비개발자를 위한 워크북 시작 안내서

내 일을 아는 AI 만들기

설치부터 AI 코치와의 첫 대화까지.
그 뒤는 내 컴퓨터의 워크북 폴더가 이어 갑니다

순원 지음 · 2026년 10월판

차례

1부 · 왜 폴더인가

1장 · AI가 뻔한 답을 하는 이유는 내 일을 모르기 때문입니다

2장 · 대화창에 쌓은 맥락은 내 것이 아닙니다

3장 · 끝나면 세 가지가 남습니다

2부 · 준비

4장 · 설치는 파일 하나를 실행하면 됩니다

5장 · 하나씩 직접 설치할 때는 이 명령을 씁니다

6장 · 로그인은 두 번입니다

7장 · GitHub는 자동으로 동기화되지 않습니다

3부 · 워크북 폴더 쓰는 법

8장 · Orca에서 폴더를 열고 코치를 부릅니다

9장 · 코치와 이렇게 진행합니다

10장 · 코치에게는 이렇게 말합니다

부록

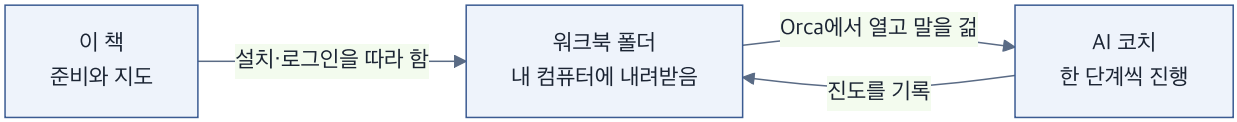
부록 A · 막혔을 때

부록 B · 용어

부록 C · 완료 체크리스트

이 책을 쓰는 법

이 책만 읽어서는 진행되지 않습니다. 책은 지도이고, 실제 수업은 내 컴퓨터에 내려받는 **워크북 폴더** 안에서 AI가 진행합니다.



	이 책	워크북 폴더
무엇인가	지금 읽는 PDF	수업 내용과 진도 파일이 든 폴더. 설치 도우미가 내려받습니다
여기서 하는 일	설치하고, 로그인하고, 워크북 폴더 쓰는 법을 익힙니다	AI 코치와 대화하며 여섯 걸음을 실제로 밟습니다
쓰는 때	처음 한 번 따라 하고, 막힐 때 다시 폅니다	8장에서 연 뒤로 끝까지

워크북 폴더에는 '바이브 코치'라는 역할이 심어져 있습니다. 폴더를 열고 말을 걸면 AI가 진도를 확인하고 한 번에 한 가지씩 이끕니다. 설치 명령과 파일 작업은 코치가 대신 합니다.

현직 개발자가 임상심리, 건축공학 같은 비개발 분야 지인 여섯 명을 직접 가르치며 막힌 곳을 고친 과정입니다.

함께 드리는 파일

함께 드리는 파일	쓰는 때
`install-mac.command`	맥에서 4장 설치를 한 번에 할 때
`install-windows.bat` · `install-windows.ps1`	윈도우에서 4장 설치를 한 번에 할 때
워크북 폴더 (설치 도우미가 내려받음)	8장부터 끝까지

읽는 순서. 1부는 10분이면 읽습니다. 2부는 컴퓨터 앞에서 따라 합니다. 3부는 워크북 폴더를 열면서 읽습니다. 막히면 부록 A를 폅니다.

걸음마다 무엇을 어떻게 하는지는 이 책에 싣지 않았습니다. 워크북 폴더에 들어 있고, 코치가 한 단계씩 꺼내 줍니다.

코치는 이렇게 움직입니다

순서	코치가 읽는 것	거기서 아는 것
1	`상태.json`	지금 몇 번째 걸음인지
2	`진행상태.md`	어디까지 했고 다음 할 일이 무엇인지
3	현재 걸음의 파일 하나	이번에 무엇을 어떻게 할지

- `진행상태.md`의 첫 번째 빈 네모가 지금 할 일입니다.
- "다 됐어요"라고 해도 코치는 화면을 확인한 뒤에만 네모를 채웁니다.
- 다음 날 다시 열고 "이어서 하자"라고 말하면 멈춘 곳에서 시작합니다.

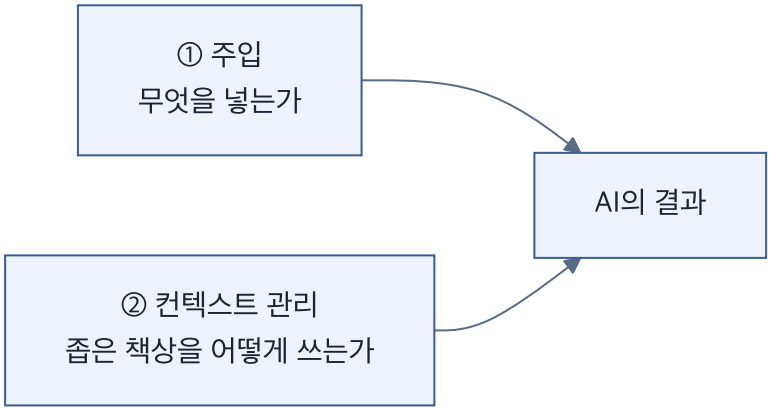
1부 · 왜 폴더인가

1장 · AI가 뻔한 답을 하는 이유는 내 일을 모르기 때문입니다

ChatGPT에 보고서 초안을 부탁하면 그럴듯한 글이 나옵니다. 그런데 우리 기관 양식도, 지난번에 반려된 이유도, 내가 싫어하는 표현도 들어 있지 않습니다. 결국 다시 씁니다.

모델이 부족해서가 아닙니다. 세상 어디에도 적혀 있지 않은 내 사정을 AI가 알 방법이 없기 때문입니다.

AI를 쓸 때 사람이 실제로 조작하는 것은 둘뿐입니다.



조작	뜻	안 하면 생기는 일
주입	내 기준, 지난 결정과 이유, 실패 기록을 글로 꺼내 읽히기	늘 일반론이 나옵니다
컨텍스트 관리	지금 일에 필요한 것만 책상에 올리기	아까 정한 것을 잊고, 중요한 지시가 묻힙니다

AI는 기억하지 않습니다. 매번 대화 전체를 처음부터 다시 읽습니다. 그 책상은 좁아서, 넘치면 오래된 것부터 밀려납니다.

결과가 나쁠 때는 이 순서로 의심합니다.

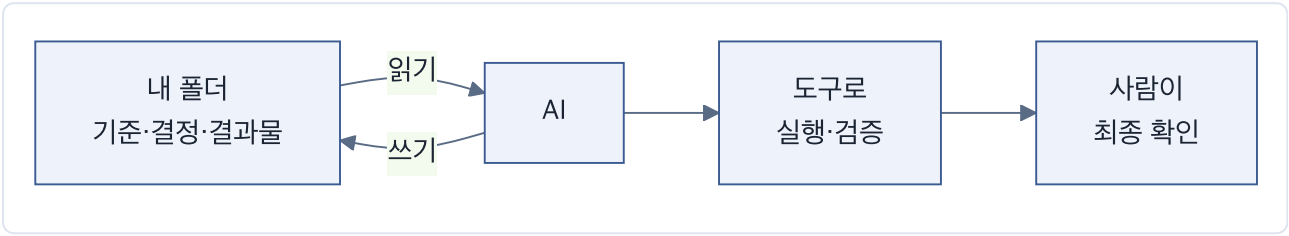
1. 재료를 안 줬는가
2. 책상이 어질러졌는가
3. 확인하지 않고 믿었는가

대부분 1번에서 끝납니다. 이 책은 1번과 2번을 폴더로 해결합니다.

2장 · 대화창에 쌓은 맥락은 내 것이 아닙니다

웹 채팅의 메모리는 플랫폼이 갖습니다. 결과물은 내가 복사해 옮겨야 남습니다. AI를 바꾸면 처음부터 다시 설명해야 합니다.

폴더는 반대입니다. 내 기준과 결정이 내 컴퓨터의 파일로 있고, AI가 일하면서 그 폴더에 직접 씁니다. 쓸수록 쌓이고 AI를 바꿔도 남습니다.



	웹 채팅	내 폴더
맥락의 주인	플랫폼	나
결과물	복사해 붙여야 남음	AI가 파일로 직접 남김
AI를 바꾸면	처음부터 다시	폴더는 그대로
보는 범위	내가 올린 파일만	연 폴더 전체

그래서 이 과정은 일반 채팅창 대신, 내 폴더를 직접 읽고 고치는 AI 도구(Claude Code 또는 Codex CLI)를 씁니다. 화면은 Orca라는 앱 하나로 통일합니다.

3장 · 끝나면 세 가지가 남습니다

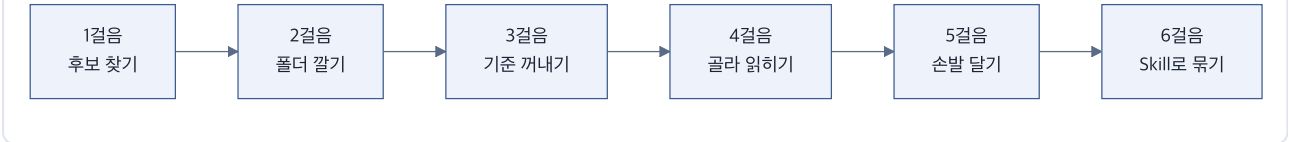
1. **내 분야 폴더.** 일하며 내린 결정과 이유, 실수와 교훈, 자주 쓰는 양식이 쌓이는 곳입니다.
2. **분야 규칙 파일.** 자주 하는 일, 잘된 결과의 기준, 용어, 하지 말 것을 적은 한 장입니다. AI가 그 폴더의 일을 할 때마다 읽습니다.
3. **자동화 하나.** 매주 되풀이하던 일 하나를 AI가 내 기준으로 처리합니다.

2번이 어떻게 생겼는지 미리 보면 이렇습니다. 워크북에 들어 있는 예시입니다.

칸	임상심리	건축공학
분야·역할	임상심리 수련생	건축공학과 3학년
지금 맡은 일	대학병원 수련, 주 3건 심리평가	졸업설계 구조계산
자주 하는 일과 결과물	심리평가 → 기관 양식 평가 보고서	구조 검토 → 계산서와 도면 코멘트
잘된 결과의 기준	검사 결과가 진단 소견으로 이어지는 근거가 보인다	적용 기준 조항을 함께 적는다
따르는 기준	DSM-5-TR	건축법·KDS
하지 말 것	내담자 실명·생년월일·기관명을 쓰지 않는다. 가명·이니셜로	기준 조항을 기억으로 지어내지 않는다. 못 찾으면 모른다고 말한다

이 한 장이 있으면 AI는 보고서를 쓸 때 기관 양식과 내가 보는 기준부터 읽습니다. 내용은 코치가 한 질문씩 물어 채웁니다.

거기까지 가는 길은 여섯 걸음이고, 워크북 폴더 속 코치가 이끕니다.



하루에 끝내지 않아도 됩니다. 컴퓨터를 껐다 켜도 코치가 진도 파일을 읽고 이어서 이끕니다.

2부 · 준비

준비는 한 시간쯤 걸립니다. 필요한 것은 여덟 가지입니다.

무엇	하는 일	꼭 필요한가
Git	잘 되던 순간을 저장점으로 남깁니다	필수
Homebrew(맥) · **winget**(윈도우)	다른 프로그램을 명령 한 줄로 설치해 줍니다	권장
Orca	폴더, 문서, AI 대화, 수정 내용을 한 창에서 보는 작업 화면	필수
GitHub 연결 도구(gh)	내 폴더를 GitHub에 백업합니다	2걸음에서 필요
Node.js	AI에게 외부 도구를 붙일 때 씁니다	5걸음에서 필요
Claude Code 또는 Codex CLI	폴더를 읽고 파일을 만들거나 고치는 AI. 둘 중 하나만	필수
GitHub 로그인과 SSH 키	비밀번호를 매번 치지 않고 GitHub에 올리게 합니다	2걸음에서 필요
워크북 폴더	코치와 진도 파일이 든 폴더. 내 GitHub 계정으로 fork해서 받습니다	필수

AI는 이미 결제했거나 익숙한 쪽을 고릅니다. Claude를 쓰면 Claude Code, ChatGPT를 쓰면 Codex CLI입니다. Claude Code는 무료 요금제로는 쓸 수 없고 Pro 이상이 필요합니다.

Claude Code와 Codex CLI는 Homebrew나 Node.js 없이도 설치됩니다. Homebrew와 Node.js는 나머지 도구를 위해 깝니다.

4장 · 설치는 파일 하나를 실행하면 됩니다

설치 도우미는 위 여덟 가지를 순서대로 **검사하고, 없는 것만 설치합니다.** 이미 Homebrew나 Codex가 깔려 있으면 건너뜁니다. 여러 번 실행해도 됩니다.

맥

1. `Command + Space`를 누르고 `터미널`을 입력해 엽니다.
2. `bash`를 입력하고 한 칸 띄웁니다.
3. `install-mac.command` 파일을 터미널 창에 끌어다 놓고 Enter를 누릅니다.

```
$ bash /Users/내이름/Downloads/install-mac.command

내 일을 아는 AI 만들기 — 설치를 시작합니다

1/8  어떤 AI를 쓰시나요?
  1) Claude Code  — Claude를 결제했거나 익숙한 분
  2) Codex CLI    — ChatGPT를 결제했거나 익숙한 분
  번호를 누르고 Enter (기본 1):
```

중간에 네 가지를 물을 수 있습니다.

화면에 나오는 것	할 일
'명령어 라인 개발자 도구' 설치 창	[설치]를 누르고, 끝나면 같은 명령을 한 번 더 실행합니다
`Press RETURN/ENTER to continue`	Enter를 누릅니다 (Homebrew 설치)
`Password:`	맥 로그인 비밀번호를 입력하고 Enter. 글자가 안 보여도 입력되고 있습니다
`GitHub 계정이 필요합니다`	계정이 있으면 1, 없으면 2를 누릅니다. 1을 고르면 6장의 GitHub 연결이 이어집니다

파일을 더블클릭해도 실행되지만, 맥이 "확인되지 않은 개발자"라며 막을 수 있습니다. 위 방법은 막히지 않습니다.

윈도우

1. `install-windows.bat`과 `install-windows.ps1`을 같은 폴더에 둡니다.
2. `install-windows.bat`을 더블클릭합니다.
3. "Windows의 PC 보호" 창이 뜨면 [추가 정보] → [실행]을 누릅니다.
4. "이 앱이 디바이스를 변경하도록 허용하시겠어요?" 창은 [예]를 누릅니다.
5. Git이나 Orca의 설치 창이 따로 뜨면 [Next]와 [Install]을 눌러 끝까지 진행합니다.

winget이 없는 윈도우에서는 도우미가 설치 파일을 직접 내려받아 실행합니다.

끝나면 이렇게 보입니다

```
끝났습니다
  ✅ Git
  ✅ Homebrew
  ✅ Orca
  ✅ GitHub 연결 도구(gh)
  ✅ Node.js
  ✅ Claude Code
  ✅ GitHub 연결(SSH)
  ✅ 워크북 폴더(fork)

이제 직접 하실 일 두 가지
  ① 터미널을 새로 열고 claude 를 입력한 뒤 Enter → 책 6장을 보며 로그인
  ② Orca를 열고 Add Repo → 아래 폴더 선택 → 폴더 옆 + 로 Claude Code 실행
     /Users/내이름/Documents/vibe-coding-curriculum
```

(윈도우는 `[남음]`) 표시가 있으면 그 항목만 5장의 명령으로 설치합니다. 여러 번 실행해도 되니, 고친 뒤 도우미를 다시 돌려도 됩니다.

AI 도구 설치가 실패로 나오면 홈 폴더의 `ai-install-log.txt`에 이유가 적혀 있습니다. 이 파일을 코치나 판매자에게 보내면 원인을 찾을 수 있습니다.

5장 · 하나씩 직접 설치할 때는 이 명령을 씁니다

도우미가 멈췄거나 직접 하고 싶을 때 봅니다. 위에서부터 한 줄씩 실행합니다.

확인 명령을 먼저 실행합니다. 버전 숫자가 나오면 이미 있는 것이니 다음 항목으로 갑니다. `command not found`(맥)나 "인식되지 않습니다"(윈도우)가 나올 때만 설치 명령을 실행합니다.

명령은 바뀔 수 있습니다. 실패하면 코치에게 "공식 문서의 최신 설치 명령을 확인해 줘"라고 부탁합니다.

맥 — 터미널에 붙여넣습니다

1. Git

```
git --version             # 확인
xcode-select --install    # 없을 때. 창이 뜨면 [설치]
```

2. Homebrew

```
brew --version            # 확인
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

설치가 끝났는데도 `brew: command not found`가 나오면 아래 두 줄을 실행합니다.

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

3. Orca

```
ls /Applications | grep Orca             # 확인. Orca.app이 나오면 있음
brew install --cask stablyai/orca/orca   # 없을 때
```

Homebrew 없이 받으려면 이 장 끝의 '직접 내려받을 때'를 봅니다.

4. GitHub 연결 도구

```
gh --version              # 확인
brew install gh           # 없을 때. 로그인은 6장
```

5. Node.js (npm이 함께 깔립니다)

```
node -v && npm -v         # 확인
brew install node         # 없을 때
```

6. AI 도구 — 하나만

```
claude --version          # Claude Code 확인
curl -fsSL https://claude.ai/install.sh | bash

codex --version           # Codex CLI 확인
curl -fsSL https://chatgpt.com/codex/install.sh | sh
```

설치했는데 `command not found`가 나오면 터미널을 닫았다가 다시 엽니다. 그래도 안 되면 아래를 실행합니다.

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

7. 워크북 폴더 — 6장의 GitHub 연결을 먼저 마칩니다

```
cd ~/Documents
gh repo fork sunwon12/vibe-coding-curriculum --clone --default-branch-only
```

이 한 줄이 내 GitHub 계정에 워크북 사본(fork)을 만들고 내 컴퓨터로 내려받습니다.

윈도우 — PowerShell에 붙여넣습니다

시작 메뉴에서 `PowerShell`을 검색해 엽니다. 줄 앞이 `PS C:\`로 시작하면 맞습니다. `C:\`만 보이면 명령 프롬프트(CMD)이니 닫고 다시 엽니다.

1. winget

```
winget --version          # 확인
start ms-windows-store://pdp/?productid=9NBLGGH4NNS1
```

두 번째 줄은 Microsoft Store의 '앱 설치 관리자' 화면을 엽니다. 거기서 설치하거나 업데이트하면 winget이 생깁니다.

2. Git

```
git --version             # 확인
winget install --id Git.Git -e --source winget
```

3. GitHub 연결 도구

```
gh --version              # 확인
winget install --id GitHub.cli -e --source winget
# 로그인은 6장
```

4. Node.js (npm이 함께 깔립니다)

```
node -v; npm -v           # 확인
winget install --id OpenJS.NodeJS.LTS -e --source winget
```

5. Orca

```
$u = "https://github.com/stablyai/orca/releases/latest/download/orca-windows-setup.exe"
$f = Join-Path $env:TEMP "orca-windows-setup.exe"
Invoke-WebRequest $u -OutFile $f
Start-Process $f -Wait
```

6. AI 도구 — 하나만

```
claude --version          # Claude Code 확인
irm https://claude.ai/install.ps1 | iex

codex --version           # Codex CLI 확인
irm https://chatgpt.com/codex/install.ps1 | iex
```

7. 워크북 폴더 — 6장의 GitHub 연결을 먼저 마칩니다

```
cd ([Environment]::GetFolderPath("MyDocuments"))
gh repo fork sunwon12/vibe-coding-curriculum --clone --default-branch-only
```

설치 직후 명령을 찾지 못하면 PowerShell 창을 닫았다가 다시 엽니다. 윈도우는 새 창에서만 방금 설치한 프로그램을 알아봅니다.

공식 스크립트가 실패할 때의 다른 길

Homebrew, winget, npm으로도 설치할 수 있습니다. npm은 Node.js가 있어야 합니다.

```
# Claude Code
brew install --cask claude-code              # 맥
winget install Anthropic.ClaudeCode          # 윈도우
npm install -g @anthropic-ai/claude-code     # 맥·윈도우

# Codex CLI
brew install --cask codex                    # 맥
npm install -g @openai/codex                 # 맥·윈도우
```

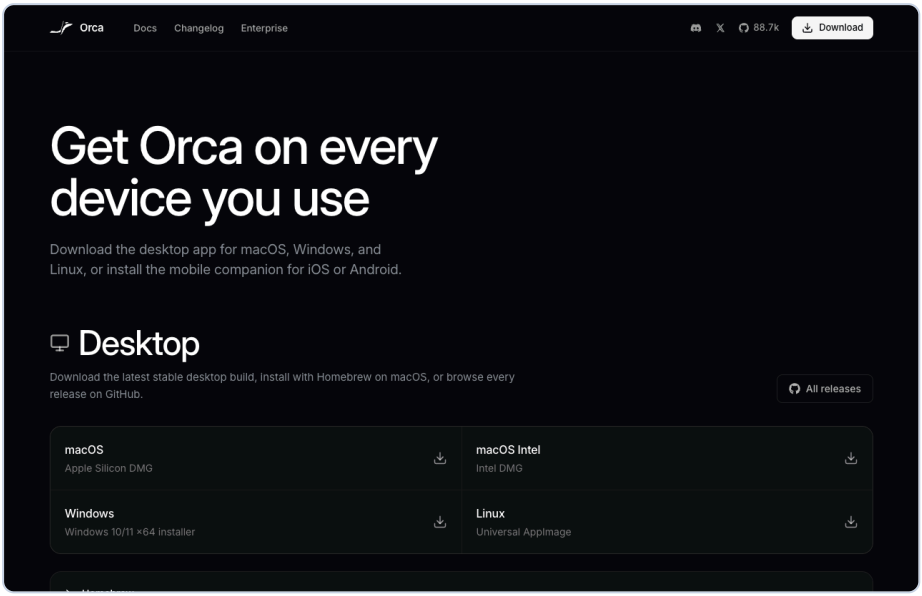
자주 나오는 오류

화면에 나온 말	원인	할 일
`command not found: brew`	Homebrew가 없거나 경로 미등록	위 2번의 명령
`command not found: claude`	설치 폴더가 경로에 없음	위 6번 아래의 두 줄
`'irm'은(는) 내부 또는 외부 명령…이 아닙니다`	CMD에서 실행함	PowerShell을 엽니다
`이 시스템에서 스크립트를 실행할 수 없으므로…`	윈도우 실행 정책	표 아래의 명령을 실행한 뒤 다시
`winget`을 찾을 수 없음	앱 설치 관리자가 오래됨	윈도우 1번의 명령
`npm: command not found`	Node.js가 없음	5번(맥) · 4번(윈도우)의 명령

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

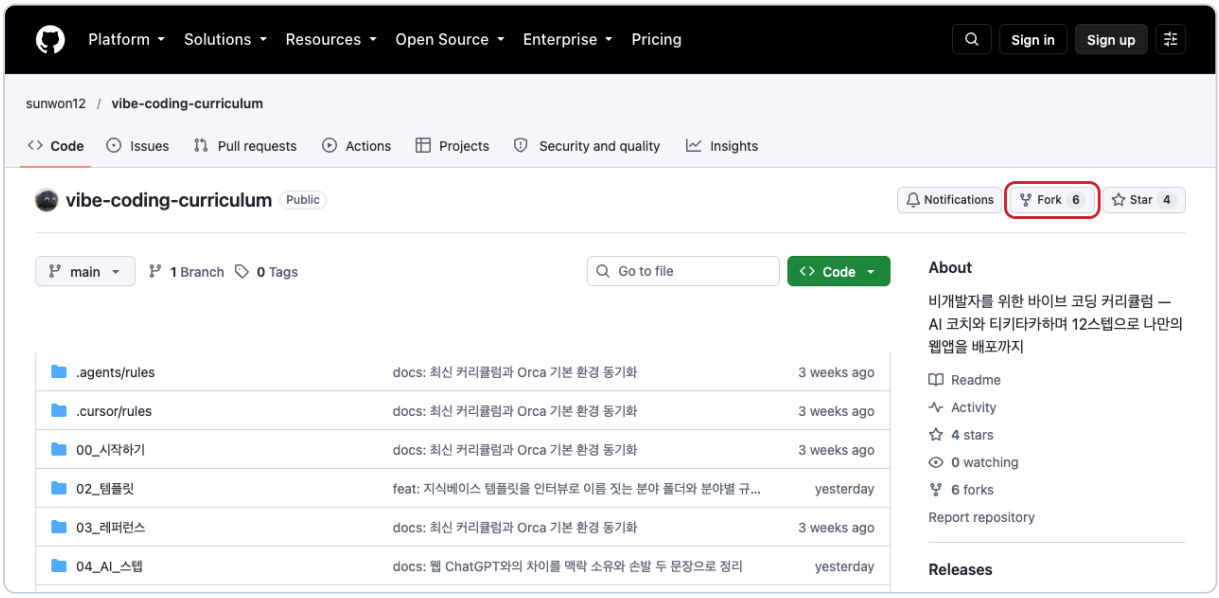
직접 내려받을 때

Orca. `onorca.dev/download`를 열고 내 컴퓨터에 맞는 파일을 받아 설치합니다.



onorca.dev/download — 맥은 macOS, 윈도우는 Windows를 누릅니다.

워크북 폴더. 웹에서 fork할 때는 `github.com/sunwon12/vibe-coding-curriculum`을 열고 오른쪽 위 **Fork** → **Create fork**를 누릅니다. 내 계정에 사본이 생기면 터미널에서 내려받습니다.



빨간 테두리가 Fork 버튼입니다. 누르려면 GitHub에 로그인돼 있어야 합니다.

```
gh repo clone 내아이디/vibe-coding-curriculum
```

ZIP으로는 받지 않습니다. ZIP으로 받은 폴더는 GitHub와 연결이 없어서 진도를 올릴 수도, 업데이트를 받을 수도 없습니다.

6장 · 로그인은 두 번입니다

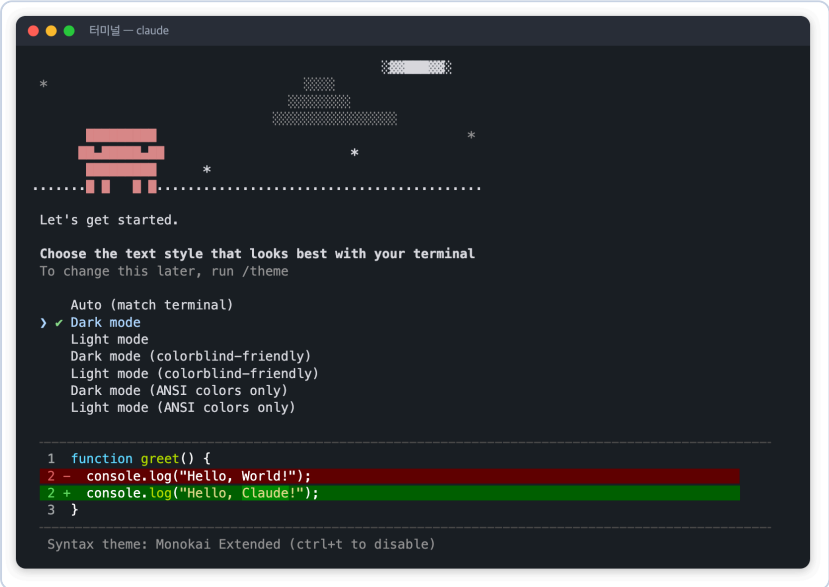
설치만으로는 연결되지 않습니다. AI 도구와 GitHub에 각각 한 번씩 로그인합니다.

로그인	필요한 계정	하는 곳
Claude Code	Claude Pro, Max, Team, Enterprise 중 하나	터미널에 `claude`
Codex CLI	ChatGPT Plus, Pro, Business, Enterprise 중 하나	터미널에 `codex`
GitHub	무료 계정	설치 도우미의 7단계, 또는 아래 명령

아래 터미널 화면은 2026년 10월에 실제 프로그램을 실행해 옮긴 것입니다. 버전이 바뀌면 문구가 조금 다를 수 있습니다.

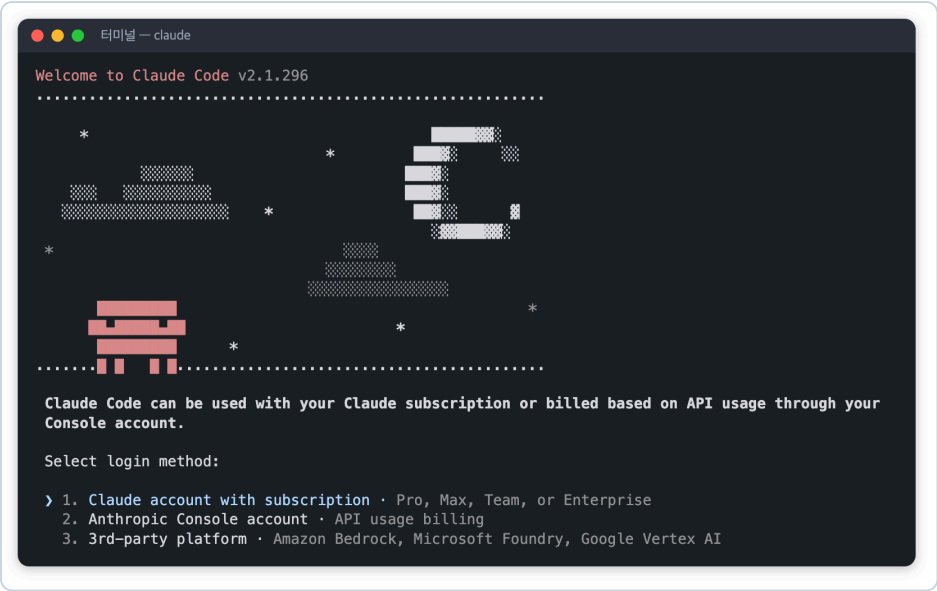
Claude Code에 로그인하기

1. 터미널을 새로 열고 `claude`를 입력한 뒤 Enter를 누릅니다. 글자 색을 고르는 화면이 나오면 그대로 Enter를 누릅니다.



처음 실행하면 나오는 화면. `Dark mode`에 표시가 있는 상태에서 Enter.

2. 로그인 방법을 묻습니다. 1번 `Claude account with subscription`에 화살표가 있는 상태에서 Enter를 누릅니다.



구독 계정으로 쓰는 경우 1번입니다.

3. 브라우저가 열리고 Claude 로그인 화면이 나옵니다. 평소 쓰는 방법(Google, 이메일)으로 로그인하고 [승인]을 누릅니다.



브라우저에 열리는 화면.

4. 브라우저가 열리지 않으면 터미널에 주소가 나와 있습니다. `c`를 누르면 주소가 복사되니 브라우저 주소창에 붙여넣습니다. 브라우저가 코드를 보여 주면 복사해 터미널의 `Paste code here if prompted >` 뒤에 붙여넣고 Enter를 누릅니다.



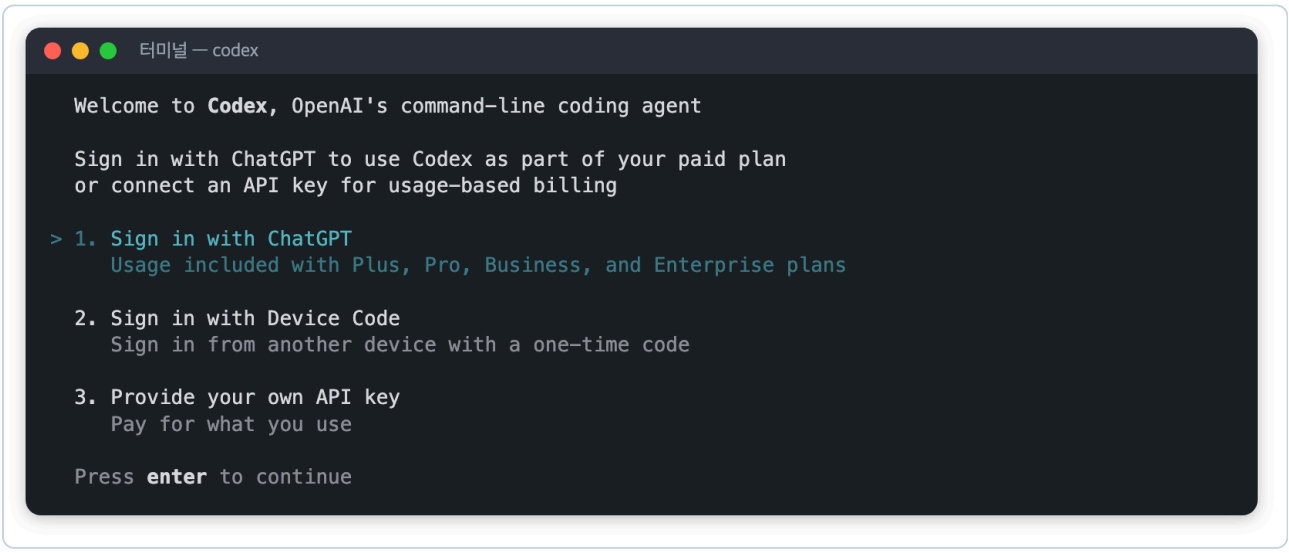
브라우저가 자동으로 안 열렸을 때의 터미널.

5. 로그인됐다는 문구가 나오면 Enter를 눌러 넘어갑니다. 이어지는 안내도 읽고 Enter를 누르면 입력창이 나옵니다.

나중에 계정을 바꾸거나 다시 로그인할 때는 Claude Code 입력창에 `/login`을 입력합니다. 로그아웃은 `/logout`입니다.

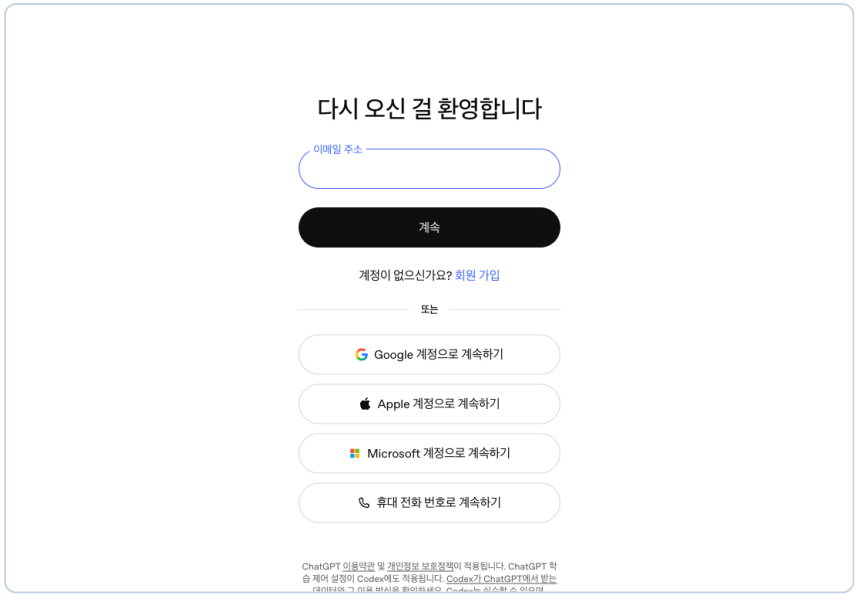
Codex CLI에 로그인하기

1. 터미널을 새로 열고 `codex`를 입력한 뒤 Enter를 누릅니다. 로그인 방법을 묻는 화면이 나옵니다. 1번 `Sign in with ChatGPT`에 화살표가 있는 상태에서 Enter를 누릅니다.



ChatGPT 유료 계정으로 쓰는 경우 1번입니다.

2. 브라우저가 열리고 ChatGPT 로그인 화면이 나옵니다. 평소 쓰는 방법(이메일, Google, Apple)으로 로그인하고 [계속]을 누릅니다.



브라우저에 열리는 화면. ChatGPT를 쓸 때와 같은 계정으로 로그인합니다.

3. 브라우저가 열리지 않으면 터미널에 주소가 나와 있습니다. `c`를 누르면 주소가 복사되니 브라우저 주소창에 붙여넣습니다.

```
터미널 — codex

  Welcome to Codex, OpenAI's command-line coding agent

  Finish signing in via your browser

  If the link doesn't open automatically, press c to copy it:

  https://auth.openai.com/oauth/authorize?response_type=code&client_id=app_EMoamEEZ73f0CkXaXp7hrann&redirect_uri=http%3A%2F%2F127.0.0.1%3A1455%2Fauth%2Fcallback&code_challenge=xkT2w1_43dl5fBHj237cEaY3d5lwJxa5SN09a0Dy9Ms&code_challenge_method=S256&state=V8LJfRV-WlIIGdY3zrnIapRw7c0LR-5LuRdjTnbICtg&scope=openid+profile+email+offline_access+api.connectors.read+api.connectors.invoke&id_token_add_organizations=true&codex_cli_simplified_flow=true&originator=codex-tui

  On a remote or headless machine? Press esc and choose Sign in with Device Code.

  Press esc to cancel
```

로그인을 기다리는 동안의 터미널. 취소하려면 `esc`를 누릅니다.

4. 브라우저에서 로그인을 마치면 터미널 화면이 넘어갑니다. 이어지는 안내를 읽고 Enter를 누르면 입력창이 나옵니다.

나중에 다시 로그인할 때는 터미널에 `codex login`을 입력합니다. 로그아웃은 `codex logout`입니다.

GitHub에 연결하기

설치 도우미의 7단계에서 1번을 고르면 아래가 자동으로 진행됩니다. 직접 할 때는 명령을 한 줄씩 실행합니다.

SSH 키는 내 컴퓨터가 GitHub에 "본인이 맞다"고 보여 주는 열쇠입니다. 한 번 등록하면 올릴 때마다 비밀번호를 치지 않습니다.

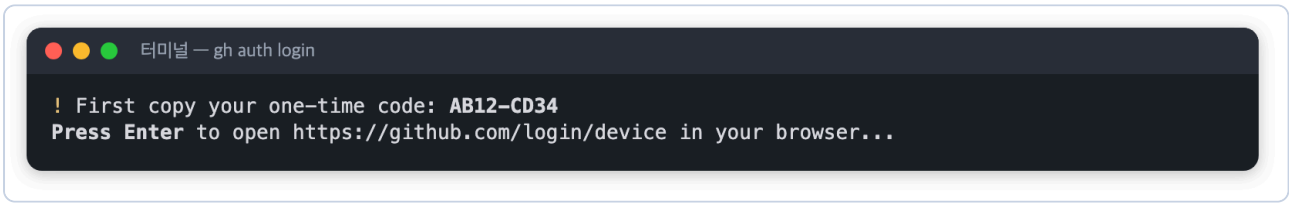
맥 터미널:

```
ls ~/.ssh/id_ed25519.pub                                # 확인. 파일이 나오면 다음 줄은 건너뜀
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519     # SSH 키 만들기
gh auth login -h github.com -p ssh -s admin:public_key -w --skip-ssh-key
gh ssh-key add ~/.ssh/id_ed25519.pub -t "내 컴퓨터"   # 키를 GitHub에 등록
ssh -T git@github.com                                   # 확인
```

윈도우 PowerShell:

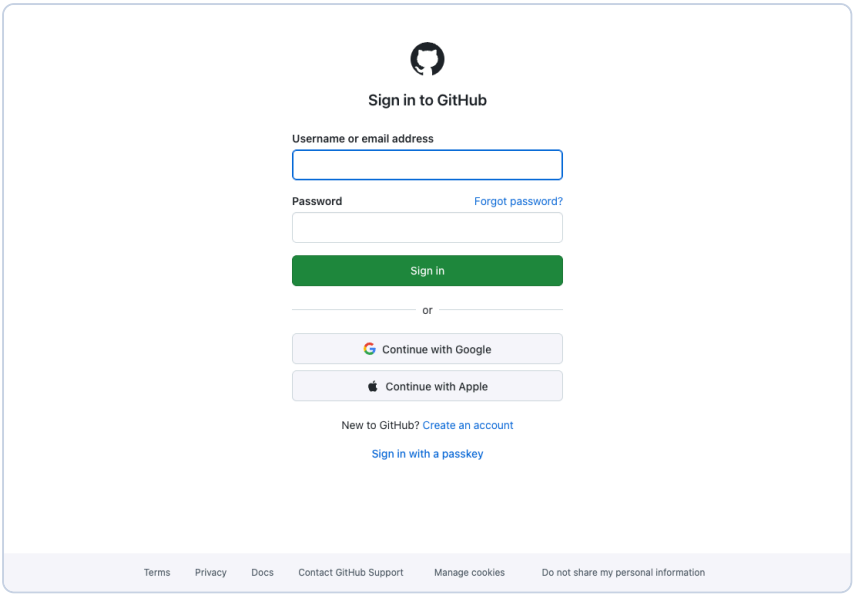
```
ls $HOME\.ssh\id_ed25519.pub                            # 확인. 파일이 나오면 다음 줄은 건너뜀
ssh-keygen -t ed25519 -f "$HOME\.ssh\id_ed25519"     # 비밀번호를 물으면 Enter를 두 번
gh auth login -h github.com -p ssh -s admin:public_key -w --skip-ssh-key
gh ssh-key add "$HOME\.ssh\id_ed25519.pub" -t "내 컴퓨터"
ssh -T git@github.com                                   # 확인
```

1. `gh auth login`을 실행하면 8자리 코드가 나옵니다. 코드를 복사하고 Enter를 누릅니다.



코드는 실행할 때마다 다릅니다.

2. 브라우저에서 GitHub에 로그인하고, 복사한 코드를 입력한 뒤 [Authorize]를 누릅니다.



계정이 없으면 아래쪽 `Create an account`로 먼저 가입합니다.

3. 마지막 줄 `ssh -T git@github.com`의 결과로 연결을 확인합니다.

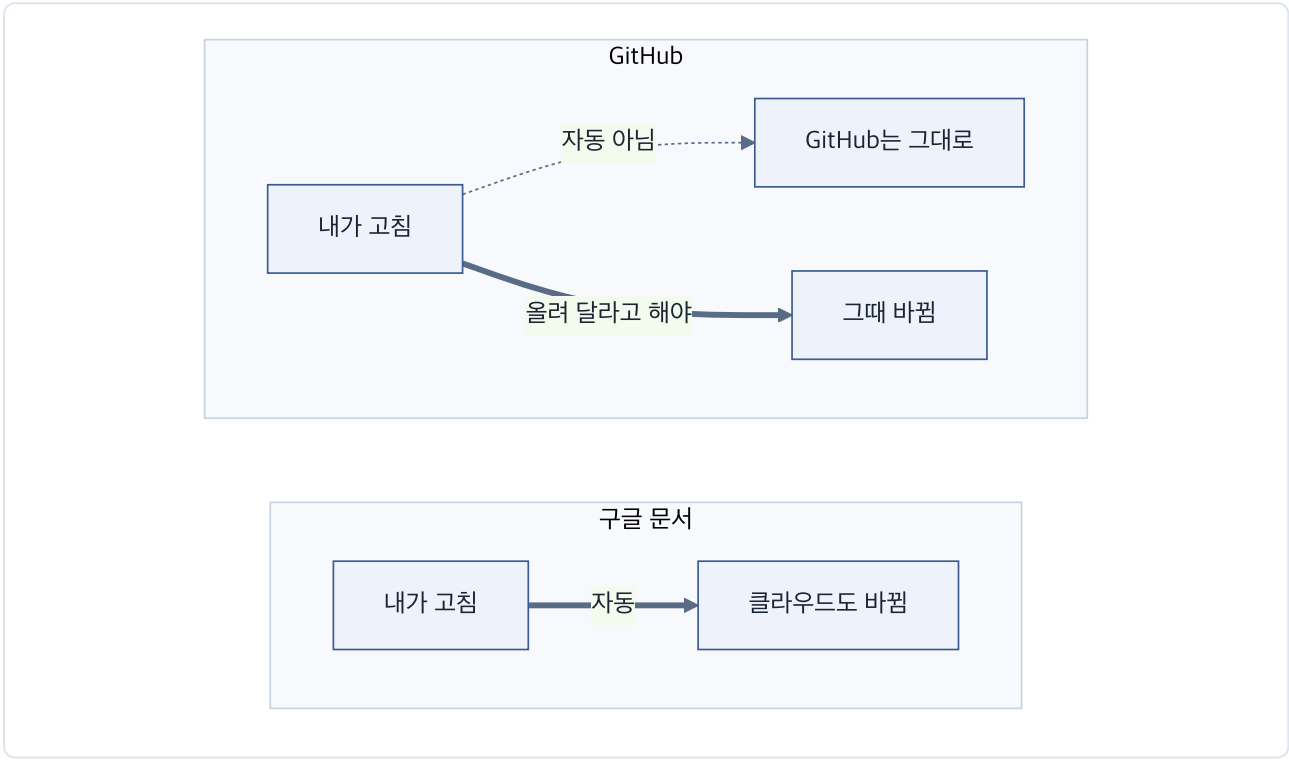
터미널에 나온 말	뜻
`Hi 내아이디! You've successfully authenticated`	연결됐습니다
`Are you sure you want to continue connecting (yes/no)?`	처음 한 번 묻습니다. `yes`를 입력하고 Enter
`Permission denied (publickey)`	키가 등록되지 않았습니다. `gh ssh-key add` 줄을 다시 실행

`.pub`가 붙지 않은 `id_ed25519` 파일은 열쇠 본체입니다. 누구에게도 보내지 않습니다.

7장 · GitHub는 자동으로 동기화되지 않습니다

강습에서 가장 많이 막힌 곳입니다. 워크북 폴더를 열기 전에 이 한 가지만 알아 둡니다.

파일은 두 곳에 따로 있습니다. 인터넷의 GitHub(원격)와 내 컴퓨터(로컬)입니다. 구글 문서와 달리 한쪽을 고쳐도 다른 쪽은 저절로 바뀌지 않습니다.



그래서 아래 두 상황은 고장이 아닙니다.

- 내 컴퓨터에서 고쳤는데 GitHub에는 그대로다 → 아직 안 올렸습니다.
- GitHub에서 바뀌었는데 내 컴퓨터는 그대로다 → 아직 안 받았습니다.

명령은 코치가 합니다. 방향만 알고 말로 시킵니다. 자세한 그림과 설명은 코치가 공통 준비에서 다시 보여 줍니다.

워크북은 내 계정으로 fork해서 받았습니다. fork는 남의 저장소를 내 계정으로 복사하는 것이라, 내 사본에는 진도를 올릴 수 있습니다. 다만 fork한 워크북은 누구나 볼 수 있는 **공개** 저장소입니다. 개인적인 내용은 워크북에 적지 않고, 2걸음에서 만드는 **비공개** 지식 폴더에 둡니다.

3부 · 워크북 폴더 쓰는 법

8장 · Orca에서 폴더를 열고 코치를 부릅니다

1. Orca를 엽니다. 홈 폴더 접근을 물으면 허용합니다.
2. 왼쪽의 **Add Repo**를 누르고 설치 도우미가 마지막에 알려 준 폴더를 고릅니다. 보통 `문서(Documents)` 안의 `vibe-coding-curriculum` 입니다.
3. 폴더 이름 옆의 **+**를 누르고 설치한 AI를 고릅니다.

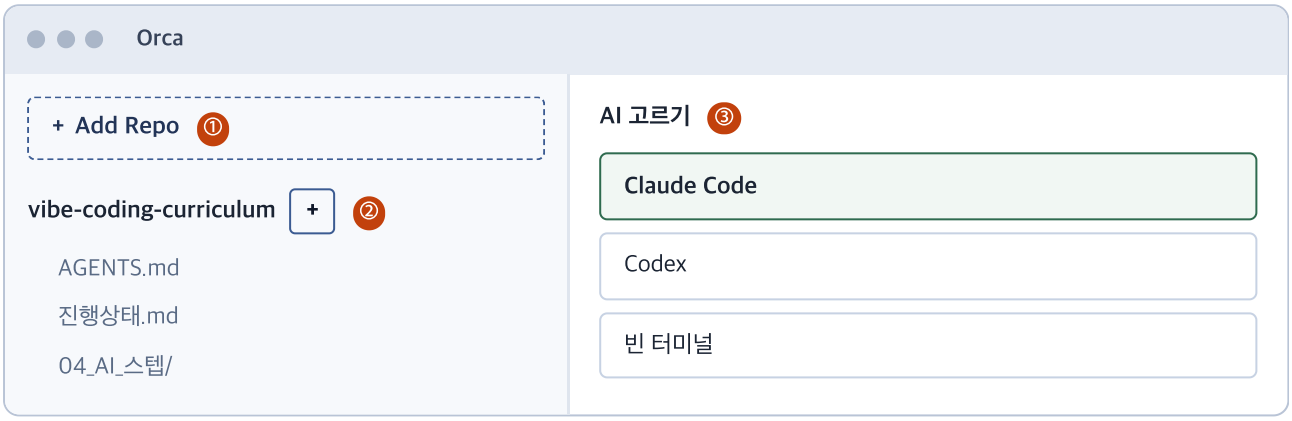


그림 — Orca 화면의 구조. ① 폴더 추가 ② 폴더 옆 + ③ AI 고르기. 실제 화면은 버전에 따라 다릅니다.

폴더가 추가되지 않으면 다른 폴더를 고른 것입니다. 설치 도우미가 마지막에 알려 준 경로를 다시 확인합니다.

허락을 매번 묻지 않게 엽니다

AI는 원래 파일을 고치거나 명령을 실행할 때마다 허락을 묻습니다. 이 과정에서는 묻지 않고 진행하는 방식으로 엽니다. 설치와 Git 작업을 코치가 대신 하는데, 질문마다 멈추면 무엇을 묻는지 판단하느라 진도가 나가지 않습니다.

Orca에서 열 때. `Settings` → `Agents` → `Agent Permissions` 가 **Yolo**인지 확인합니다. Yolo이면 폴더 옆 +로 연 AI가 이 방식으로 열립니다.

터미널에서 직접 열 때. 워크북 폴더로 이동한 뒤 아래 중 내가 쓰는 AI의 한 줄을 실행합니다.

```
claude --dangerously-skip-permissions
codex --dangerously-bypass-approvals-and-sandbox
```

처음 실행하면 경고 화면이 한 번 나올 수 있습니다. 읽고 수락을 고릅니다.

이 방식에서는 AI가 묻지 않고 파일을 고치고 명령을 실행합니다. 그래서 세 가지를 지킵니다.

1. 워크북 폴더와 내 지식 폴더에서만 이렇게 엽니다. 회사 자료나 사진처럼 잃으면 안 되는 폴더에서는 쓰지 않습니다.

2. 잘 되던 순간마다 저장점을 남깁니다. 망가지면 "바꾸기 직전으로 되돌려 줘"라고 합니다.
3. 결제, 삭제, 외부 발송, 공개가 걸린 일은 "그 직전에 멈춰서 물어봐"라고 먼저 말해 둡니다.

첫 문장

AI 창이 열리면 아래를 그대로 붙여넣습니다.

코치에게 붙여넣기

나 이 레포로 바이브 코딩 처음 배워. 완전 초보야.
AGENTS.md 규칙대로 바이브 코치가 되어서, 지금 몇 스텝인지 확인하고 처음부터 이끌어줘.

코치가 `진행상태.md`를 읽고 공통 준비부터 확인합니다. 설치가 끝났으면 금방 지나갑니다.

준비가 끝나면 코치가 과정을 묻습니다. **"AI 스텝"**이라고 답합니다. 웹앱을 만드는 프로덕트 스텝은 같은 폴더에 있는 별도 과정입니다.

AI가 "파일을 첨부해 달라"고만 한다면 다른 폴더에서 실행된 것입니다. 2번부터 다시 합니다.

코치가 답하기 시작하면 수업은 이 화면에서 이어집니다.

9장 · 코치와 이렇게 진행합니다

한 번에 한 가지씩 갑니다

코치는 한 번에 한 가지만 시킵니다. 내가 "다 됐어요"라고 해도 화면을 확인한 뒤에만 진도를 채웁니다. 안 한 것을 했다고 표시하면 나중에 길을 잃기 때문입니다.

설치와 명령, Git 작업은 코치가 대신 합니다. 무엇을 만들지와 왜 만들지는 내가 정합니다. 코치는 대신 정하지 않고 되묻습니다.

과정은 둘 중 하나를 고릅니다

공통 준비가 끝나면 코치가 묻습니다.

과정	이런 사람	끝나면 생기는 것
AI 스텝	코딩 없이 내 일을 줄이고 싶다	내 일을 내 기준으로 처리하는 AI
프로덕트 스텝	웹앱을 만들어 보고 싶다	링크로 보낼 수 있는 내 웹앱

이 책의 제목은 AI 스텝 쪽입니다. 두 과정은 같은 워크북 폴더에 들어 있고, 설치와 로그인은 똑같습니다.

AI 스텝은 여섯 걸음입니다

걸음	끝나면 생기는 것	걸리는 시간
1. 후보 찾기	입력·결과·위험이 정해진 첫 자동화 후보	20분
2. 폴더 깔기	AI가 새 대화에서도 찾아 읽는 내 지식 폴더	30분
3. 기준 꺼내기	내 기준·결정 이유·실패가 담긴 판단 재료	25분
4. 골라 읽히기	넣을 것·뺄 것·읽힐 때가 정해진 구조	30분
5. 손발 달기	파일·브라우저·업무 서비스 중 실제 연결 하나	40분
6. Skill로 묶기	다음에도 재사용되는 반복 작업 매뉴얼	30분

처음에는 1 → 2 순서를 권합니다. 그 뒤로는 막힌 이유에 따라 고릅니다.

지금 막힌 이유	갈 곳
AI가 내 사정과 기준을 모른다	3걸음
무엇을 기록하고 언제 읽힐지 모르겠다	4걸음
파일·화면·서비스를 직접 못 만진다	5걸음
매번 같은 설명과 순서를 반복한다	6걸음

멈췄다가 다시 할 때

Orca에서 폴더 옆 +로 AI를 열고 "이어서 하자"라고 말합니다. 코치가 진도 파일을 읽고 멈춘 곳에서 시작합니다. 진도를 내가 기억할 필요는 없습니다.

대화가 길어져 AI가 앞에서 정한 것을 잊기 시작하면 새 대화를 엽니다. 정한 것은 파일에 남아 있어서 잃지 않습니다.

진도는 내 GitHub에 올라갑니다

걸음을 끝낼 때마다 코치가 진도 파일 두 개를 내 fork에 올립니다. 그래서 컴퓨터를 바꿔도 이어 할 수 있습니다.

fork는 공개 저장소라 어디까지 했는지가 누구에게나 보입니다. 코치가 처음 올리기 전에 한 번 묻습니다. 원하지 않으면 "올리지 마"라고 하면 내 컴퓨터에만 남습니다.

처음에 한 번 말해 둘 것

묻지 않고 진행하는 방식으로 열었으니, 되돌리기 어려운 일은 먼저 선을 그어 둡니다.

코치에게 붙여넣기

결제, 삭제, 외부 발송, 공개가 걸린 일은 실행하기 직전에 멈춰서 나한테 물어봐.
내가 말하지 않은 내용은 지어내지 말고, 모르면 모른다고 해.

10장 · 코치에게는 이렇게 말합니다

특별한 문법은 없습니다. 평소 말로 하고, 결과가 어긋나면 한 가지만 고쳐 달라고 합니다.

진행할 때

상황	하는 말
시작하거나 이어 갈 때	"이어서 하자."
설명이 어려울 때	"글 말고 그림으로 보여 줘."
모르는 단어가 나올 때	"이 단어 무슨 뜻이야?"
화면이 예상과 다를 때	화면을 캡처해 붙이고 "지금 이렇게 보여."
빨간 글씨가 떴을 때	전체를 복사해 붙이고 "바로 고치지 말고 원인부터 설명해 줘."

결과가 마음에 안 들 때

증상	먼저 의심할 것	하는 말
뻔한 일반론이 나옴	내 기록·기준이 빠짐	"이 파일을 근거로 다시 해 줘."
엉뚱한 부분까지 바뀜	지킬 선이 빠짐	"이 부분만 바꾸고 나머지는 유지해 줘."
답이 산만함	자료나 요구가 너무 많음	"지금은 이 파일 하나와 이 결과만 다뤄 줘."
다 됐다는데 못 믿겠음	확인 방법이 빠짐	"어떻게 검증했는지 결과를 보여 줘."
같은 실수를 반복함	대화가 꼬임	새 대화를 열고 "이어서 하자."

저장하고 되돌릴 때

하고 싶은 일	하는 말
상태 확인	"이 폴더 GitHub랑 연결돼 있어? 안 올라간 변경 있어?"
저장점 남기기	"지금 상태를 저장점으로 남겨 줘."
올리기	"지금까지 바꾼 걸 저장점으로 남기고 GitHub에 올려 줘."
받기	"새로 올라온 게 있으면 받아 줘. 부딪히면 먼저 설명해 줘."
되돌리기	"방금 바꾼 것 때문에 망가졌어. 바꾸기 직전으로 되돌려 줘."

부록

부록 A · 막혔을 때

대부분은 화면이나 빨간 글씨를 그대로 코치에게 붙여넣고 "이거 왜 이래?"라고 물으면 풀립니다.

설치가 안 됩니다. 명령이 실패합니다. 설치는 원래 코치가 하는 일입니다. 실패한 내용을 통째로 주고 "왜 그런지와 어떻게 하면 되는지 알려 줘"라고 합니다. 흔한 원인은 인터넷 끊김, 프로그램 미설치, 폴더 위치입니다.

빨간 글씨가 떴습니다. 고장이 아니라 "여기를 봐 달라"는 안내입니다. 이것저것 누르지 말고 전체를 복사해 코치에게 줍니다. "바로 고치지 말고 원인부터 설명해 줘"를 붙이면 더 안전합니다.

GitHub 페이지에는 파일이 보이는데 내 컴퓨터에는 없습니다. 정상입니다. 아직 받지 않은 상태입니다. 설치 도우미를 실행하거나 코치에게 주소를 주고 "내 컴퓨터로 가져와 줘"라고 합니다.

내 컴퓨터에서 고쳤는데 GitHub에 가 보면 그대로입니다. 정상입니다. 고치기 → commit → push 중 push를 아직 안 했습니다. "저장점으로 남기고 올려 줘"라고 합니다.

AI가 엉뚱한 것을 고쳤습니다. "방금 바꾼 것 때문에 망가졌어. 바꾸기 직전으로 되돌려 줘"라고 합니다. Orca의 변경 비교 화면에서 무엇이 바뀌었는지 먼저 봅니다. 잘 되던 순간마다 저장점을 찍어 두면 언제든 돌아갑니다.

AI가 같은 실수를 반복합니다. 대화가 꼬인 것입니다. 새 대화를 열어 상황을 짧게 다시 설명하거나, 저장점으로 되돌리거나, "완전히 다른 방법으로 해 보자"고 합니다.

AI가 이건 못 한다고 합니다. 원인은 셋이고 처방이 다릅니다.

원인	예	처방
손이 없다	내 시트·브라우저를 볼 수 없다	도구를 붙입니다 (5걸음)
모른다	내 취향, 우리 팀 사정	글로 적어 읽힙니다 (3걸음)
구조적으로 못 한다	정확한 계산, 스스로 틀린 줄 알기	사람이 하거나 검증을 붙입니다

비밀 값이 걱정됩니다. 비밀번호와 열쇠 값은 파일에 직접 쓰지 않습니다. 실수로 올렸다면 그 값을 새로 발급받아 바꾸는 것이 가장 확실합니다. 코치에게 "이 키를 실수로 올렸어, 안전하게 처리해 줘"라고 합니다.

돈이 드나요? 이 과정은 AI 도구 구독 하나를 전제로 합니다. 그 밖에 결제 정보를 요구하는 화면이 나오면 멈추고 코치에게 묻습니다.

부록 B · 용어

용어	쉬운 뜻
프롬프트	AI에게 시키려고 쓰는 말
컨텍스트	AI가 답할 때 보는 책상 위 전부. 대화, 규칙 파일, 붙인 자료
컨텍스트 윈도우	그 책상의 크기. 넘치면 오래된 것이 밀려납니다
토큰	책상 위 분량을 재는 단위
compact	길어진 대화를 요약해 줄이기. 세부가 빠집니다
환각	모르는 것을 자신 있게 지어내는 것. 없앨 수 없어 확인합니다
지식베이스	내 기준과 경험을 글로 꺼내 AI가 읽게 만든 폴더
규칙 파일	AI가 폴더를 열 때 먼저 읽는 지시문. `CLAUDE.md`, `AGENTS.md`
MCP	AI에게 외부 도구를 꽂아 주는 표준 콘센트
CLI	글자로 명령하는 프로그램. Claude Code와 Codex CLI가 여기에 듭니다
에이전트	생각 → 실행 → 확인을 스스로 반복하는 AI
Skill	AI가 필요할 때 읽는 반복 작업 매뉴얼
터미널	컴퓨터에 글자로 명령을 내리는 창
Git	작업의 저장점을 관리하는 도구
GitHub	그 기록을 올려 두는 인터넷 창고
저장소(레포)	프로젝트 파일이 통째로 담긴 상자
fork	남의 저장소를 내 계정으로 복사하는 것. 내 사본에는 마음대로 올릴 수 있습니다
로컬 / 원격	내 컴퓨터의 사본 / 인터넷의 원본
clone · pull	원격에서 로컬로. 처음 통째로 / 그 뒤 바뀐 것만
commit · push	저장점 찍기(내 컴퓨터 안) / 인터넷에 올리기
충돌	같은 파일을 양쪽에서 따로 고쳐 부딪힌 상태
API 키	서비스를 쓸 수 있는 내 전용 열쇠. 남에게 보이면 안 됩니다

부록 C · 완료 체크리스트

준비

- [ ] 설치 도우미의 결과에 ✓ 나 `[남음]` 이 없습니다.
- [ ] Orca를 설치했습니다.
- [ ] Claude Code와 Codex CLI 중 하나를 설치하고 로그인했습니다.
- [ ] `ssh -T git@github.com` 에 `successfully authenticated` 가 나옵니다.
- [ ] 워크북을 내 GitHub로 fork했고, 그 폴더를 Orca에 추가했습니다.
- [ ] AI가 `AGENTS.md` 와 `진행상태.md` 를 직접 읽고 답했습니다.
- [ ] commit과 push가 다른 단계라는 것을 설명할 수 있습니다.

여섯 걸음

- [ ] 1걸음 — 첫 자동화 후보 하나의 입력·결과·확인 지점을 정했습니다.
- [ ] 2걸음 — 내 지식 폴더가 비공개 저장소로 올라갔고, 새 대화에서 내 기록이 쓰였습니다.
- [ ] 3걸음 — 결과를 바꾸는 내 기준이 세 건 이상 기록됐습니다.
- [ ] 4걸음 — 넣을 것과 뺄 것, 읽힐 때를 구분합니다.
- [ ] 5걸음 — AI가 파일·브라우저·서비스 중 하나를 직접 다뤘습니다.
- [ ] 6걸음 — Skill 하나가 새 사례에서 같은 결과를 냈습니다.